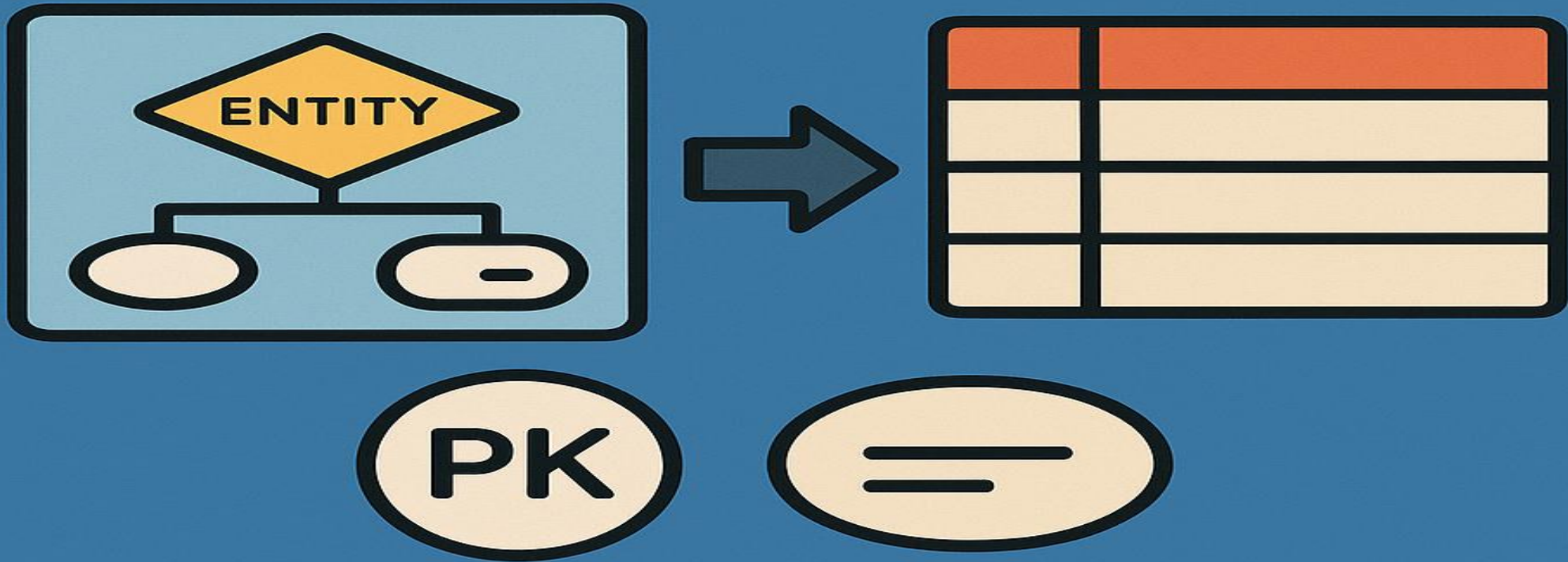


# BASIS DATA



# Mapping Entity Relationship to Relational Model

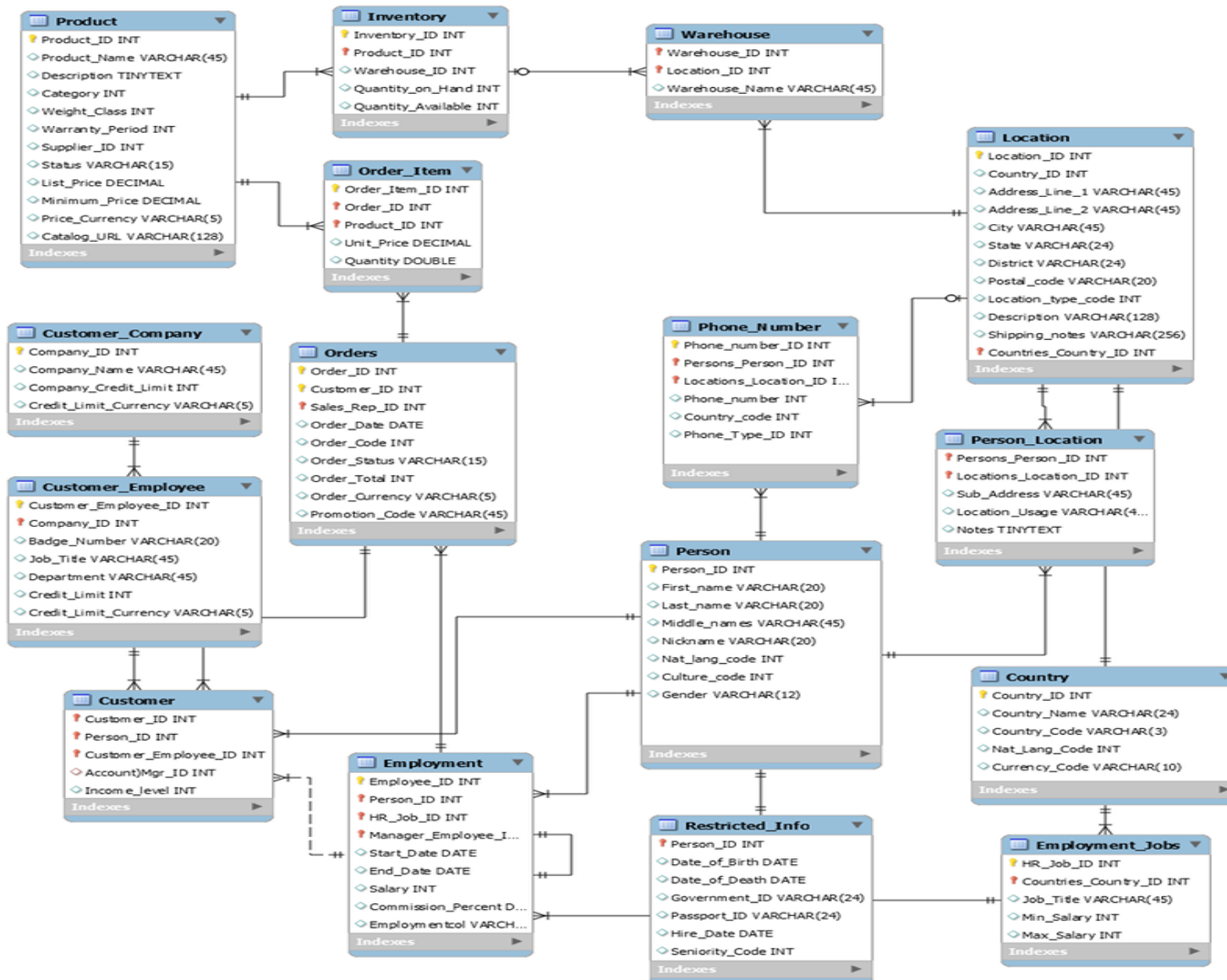


# Mapping Entity Relationship to Relational Model

- EER ? Why Need?
- Specialization
- Generalization
- Disjoint/Overlapping
- Completeness Constrains (single/double line)

# Why

Easier to be implemented in Database tools as a table and relational table



# Database Designer

- Implement real world case to be data relational in database
- How:
  1. Determined Mini World
  2. Create ER or E-ER
  - 3. Create Relational Model**

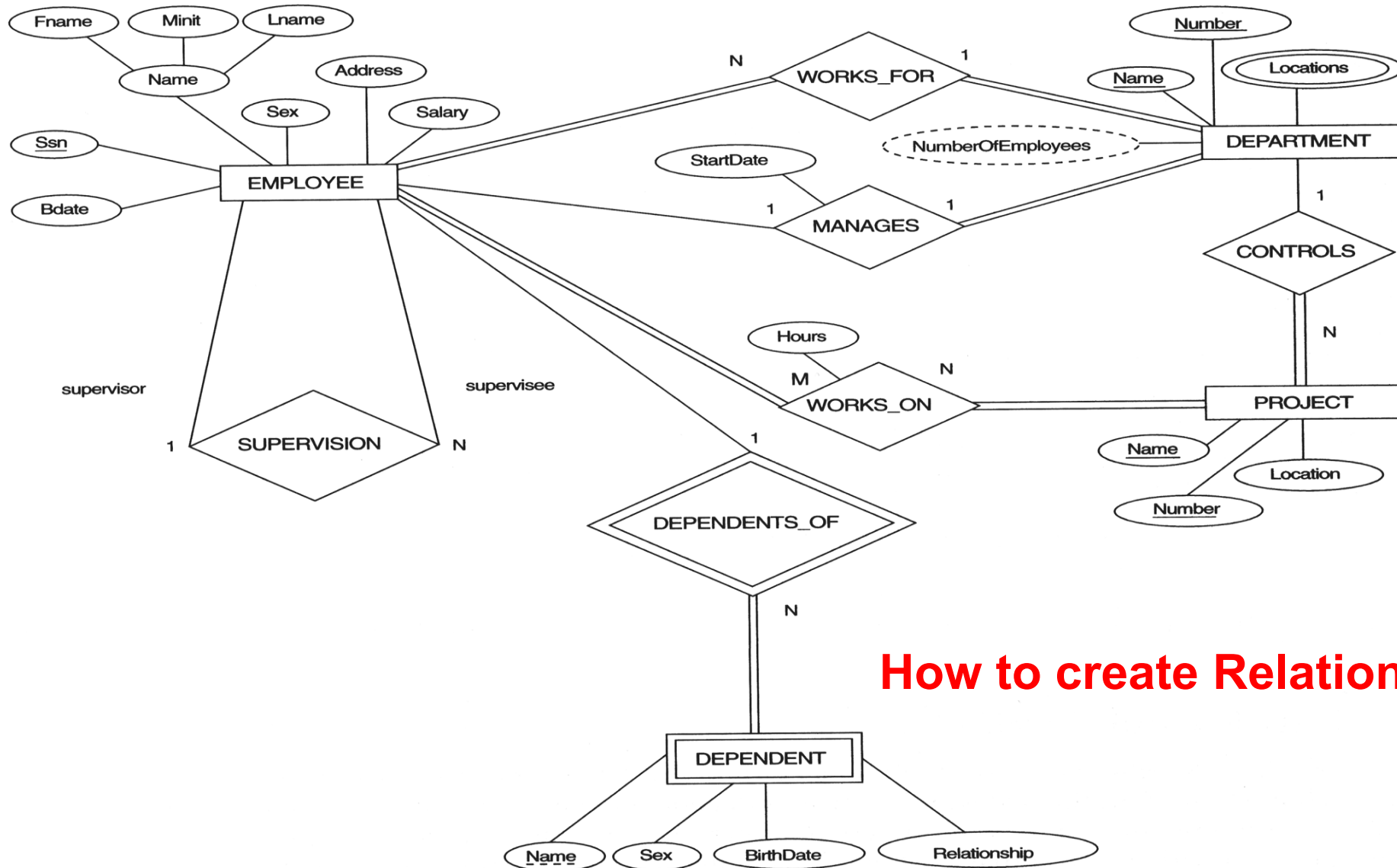
## **A. ER to Realtional Model**

1. Mapping of Regular Entity Types
2. Mapping of Weak Entity Types
3. Mapping of Binary 1:1 Relationship Types
4. Mapping of Binary 1:N Relationship Types.
5. Mapping of Binary M:N Relationship Types.
6. Mapping of Multivalued attributes.
7. Mapping of N-ary Relationship Types.

## **B. E-ER to Relational Model**

8. Options for Mapping Specialization or Generalization.
9. Mapping of Union Types (Categories).

# ER



**How to create Relational Model??**

# 1. Mapping of Regular Entity Types

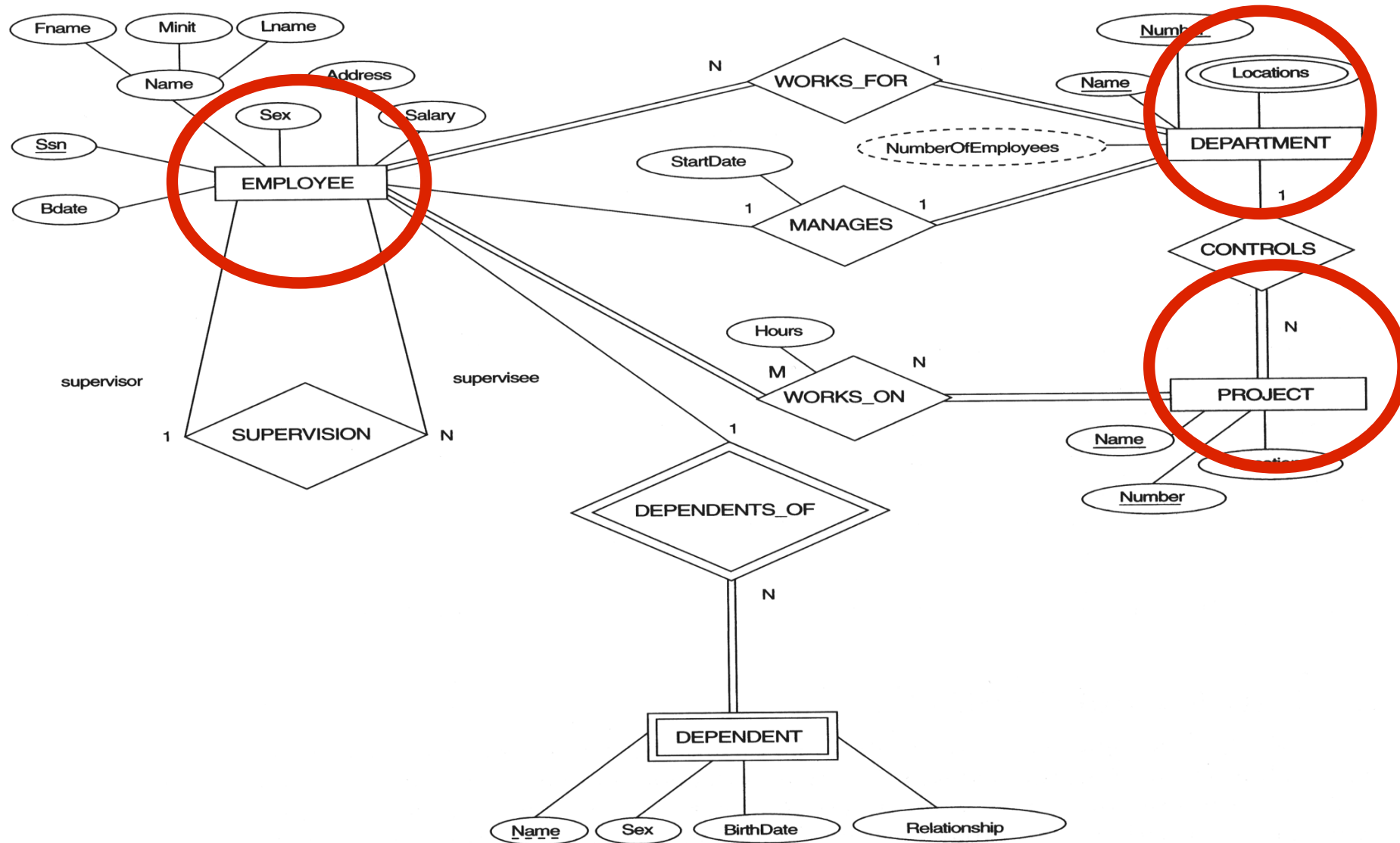
- **For each regular (strong) entity** type E in the ER schema, create a relation R that includes all the simple attributes of E.
- **Choose one of the key** attributes of E as the primary key for R. If the chosen key of E is composite, the set of simple attributes that form it will together form the primary key of R.

# 1. Mapping of Regular Entity Types (cont)

## Example:

- We create the relations EMPLOYEE, DEPARTMENT, and PROJECT, includes all the simple attributes of E in the relational schema corresponding to the regular entities in the ER diagram.
- SSN, DNUMBER, and PNUMBER are the primary keys for the relations EMPLOYEE, DEPARTMENT, and PROJECT as shown.

# 1. Mapping of Regular Entity Types (cont)



# 1. Mapping of Regular Entity Types (cont)

EMPLOYEE

|            |       |       |       |       |         |     |        |
|------------|-------|-------|-------|-------|---------|-----|--------|
| <u>Ssn</u> | Fname | Minit | Lname | Bdate | Address | Sex | Salary |
|------------|-------|-------|-------|-------|---------|-----|--------|

DEPARTMENT

|                |       |
|----------------|-------|
| <u>Dnumber</u> | Dname |
|----------------|-------|

PROJECT

|                |       |           |
|----------------|-------|-----------|
| <u>Pnumber</u> | Pname | Plocation |
|----------------|-------|-----------|

## 2. Mapping of Weak Entity Types

- For each **weak** entity type  $W$  in the ER schema with owner entity type  $E$ , **create a relation  $R$  and include all simple attributes** (or simple components of composite attributes) of  $W$  as attributes of  $R$ .
- In addition, **include as foreign key attributes of  $R$  the primary key attribute(s) of the relation(s)** that correspond to the owner entity type(s).
- The **primary key of  $R$  is the *combination of the primary key(s) of the owner(s) and the partial key of the weak entity type  $W$ , if any.***

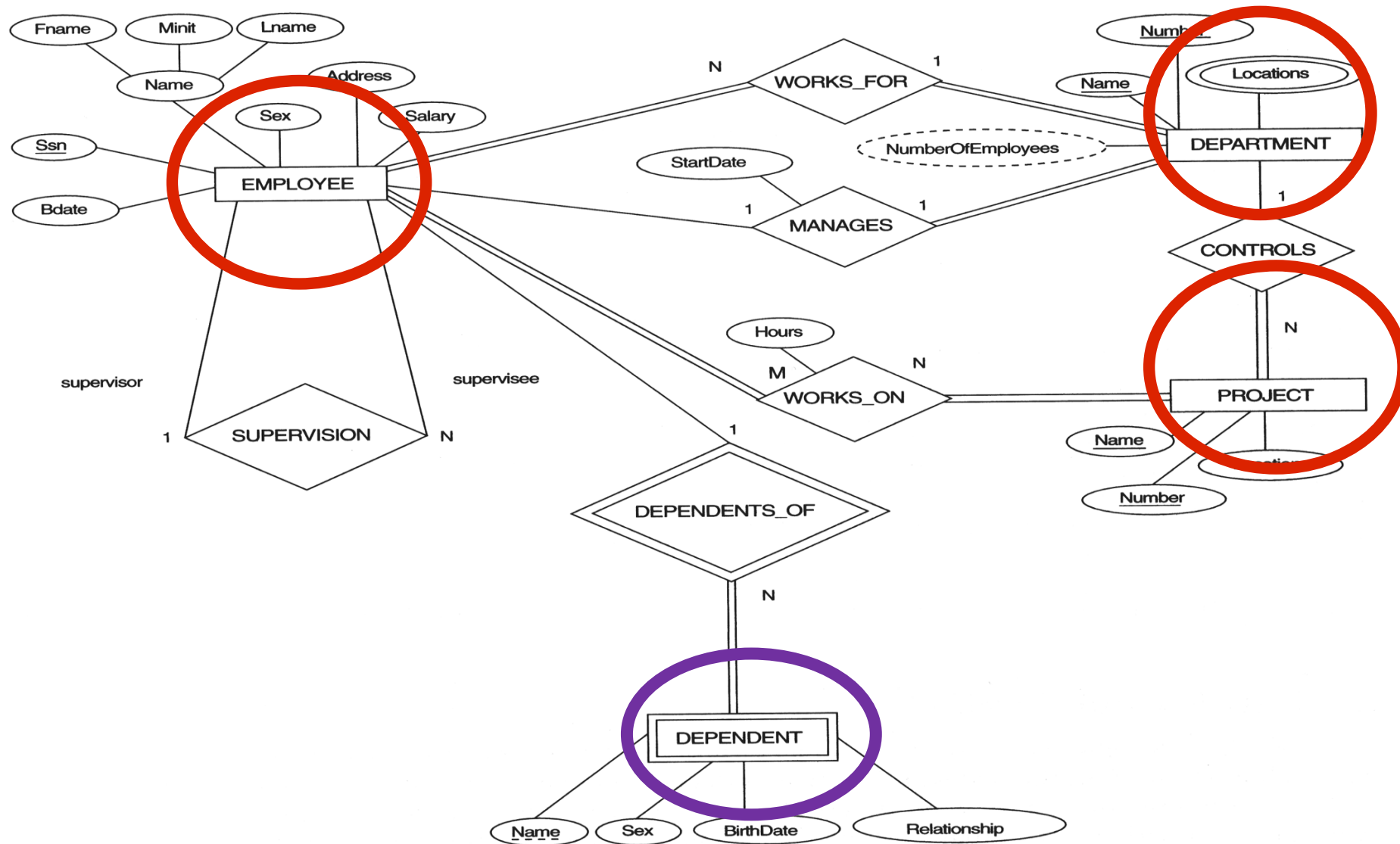
## 2. Mapping of Weak Entity Types (cont)

- **Example:**

Create the **relation DEPENDENT** in this step to correspond to the weak entity type DEPENDENT. Include the primary key SSN of the EMPLOYEE relation as a foreign key attribute of DEPENDENT (renamed to ESSN).

The **primary key** of the DEPENDENT relation is the combination {ESSN, DEPENDENT\_NAME} because DEPENDENT\_NAME is the **partial key** of DEPENDENT.

## 2. Mapping of Weak Entity Types (cont)



## 2. Mapping of Weak Entity Types (cont)

### EMPLOYEE

|            |       |       |       |       |         |     |        |
|------------|-------|-------|-------|-------|---------|-----|--------|
| <u>Ssn</u> | Fname | Minit | Lname | Bdate | Address | Sex | Salary |
|------------|-------|-------|-------|-------|---------|-----|--------|

### DEPARTMENT

|                |       |
|----------------|-------|
| <u>Dnumber</u> | Dname |
|----------------|-------|

### PROJECT

|                |       |           |
|----------------|-------|-----------|
| <u>Pnumber</u> | Pname | Plocation |
|----------------|-------|-----------|

### DEPENDENT

|             |                      |     |       |              |
|-------------|----------------------|-----|-------|--------------|
| <u>Essn</u> | <u>DependentName</u> | Sex | Bdate | Relationship |
|-------------|----------------------|-----|-------|--------------|

### 3. Mapping of Binary 1:1 Relationship Types

For each binary 1:1 relationship type  $R$  in the ER schema, identify the relations  $S$  and  $T$  that correspond to the entity types participating in  $R$ .

There are three possible approaches:

1. Foreign Key approach: Choose one of the relations- $S$ , say-and include a foreign key in  $S$  the primary key of  $T$ . It is better to choose an entity type **with total participation in  $R$  in the role of  $S$** .

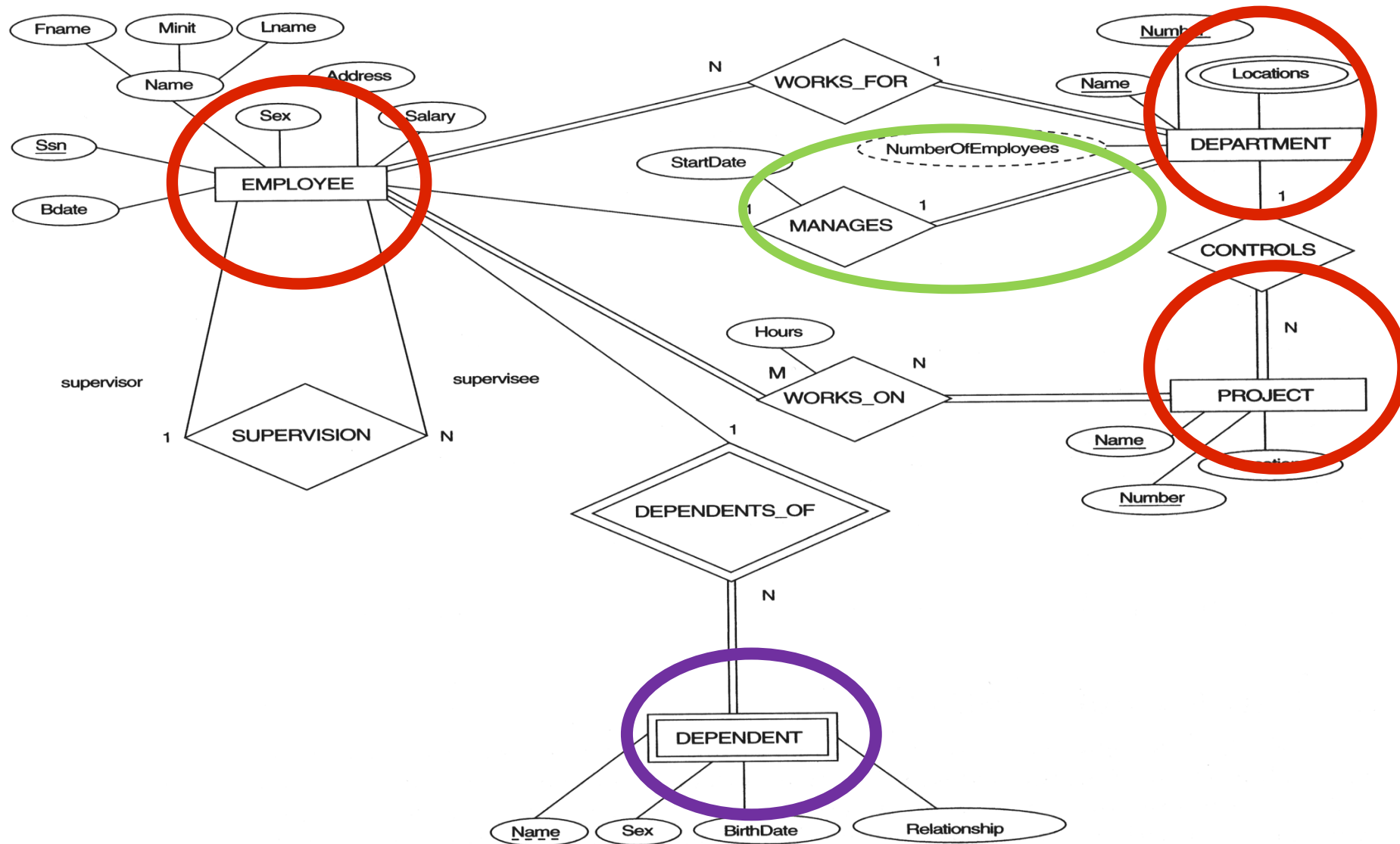
**Example:**

**1:1 relation MANAGES** is mapped by choosing the participating entity type DEPARTMENT to serve in the role of  $S$ , because its participation in the MANAGES relationship type is total.

### 3. Mapping of Binary 1:1 Relationship Types

2. Merged relation option: An alternate mapping of a 1:1 relationship type is possible by merging the two entity types and the relationship into a single relation. This may be appropriate when both participations are total.
3. Cross-reference or relationship relation option: The third alternative is to set up a third relation R for the purpose of cross-referencing the primary keys of the two relations S and T representing the entity types. This may be appropriate when both participations are partials.

# 3. Mapping of Binary 1:1 Relationship Types (cont)



### 3. Mapping of Binary 1:1 Relationship Types

#### EMPLOYEE

|            |       |       |       |       |         |     |        |
|------------|-------|-------|-------|-------|---------|-----|--------|
| <u>Ssn</u> | Fname | Minit | Lname | Bdate | Address | Sex | Salary |
|------------|-------|-------|-------|-------|---------|-----|--------|

#### DEPARTMENT

|                |       |        |              |
|----------------|-------|--------|--------------|
| <u>Dnumber</u> | Dname | Mgrssn | Mgrstartdate |
|----------------|-------|--------|--------------|

#### PROJECT

|                |       |           |
|----------------|-------|-----------|
| <u>Pnumber</u> | Pname | Plocation |
|----------------|-------|-----------|

#### DEPENDENT

|             |                      |     |       |              |
|-------------|----------------------|-----|-------|--------------|
| <u>Essn</u> | <u>DependentName</u> | Sex | Bdate | Relationship |
|-------------|----------------------|-----|-------|--------------|

## 4. Mapping of Binary 1:N Relationship Types.

- For each regular **binary 1:N relationship** type R, identify the relation S that represent the participating entity type at the N-side of the relationship type.
- Include as foreign key** in S the primary key of the relation T that represents the other entity type participating in R.
- Include any simple attributes** of the 1:N relation type as attributes of S.

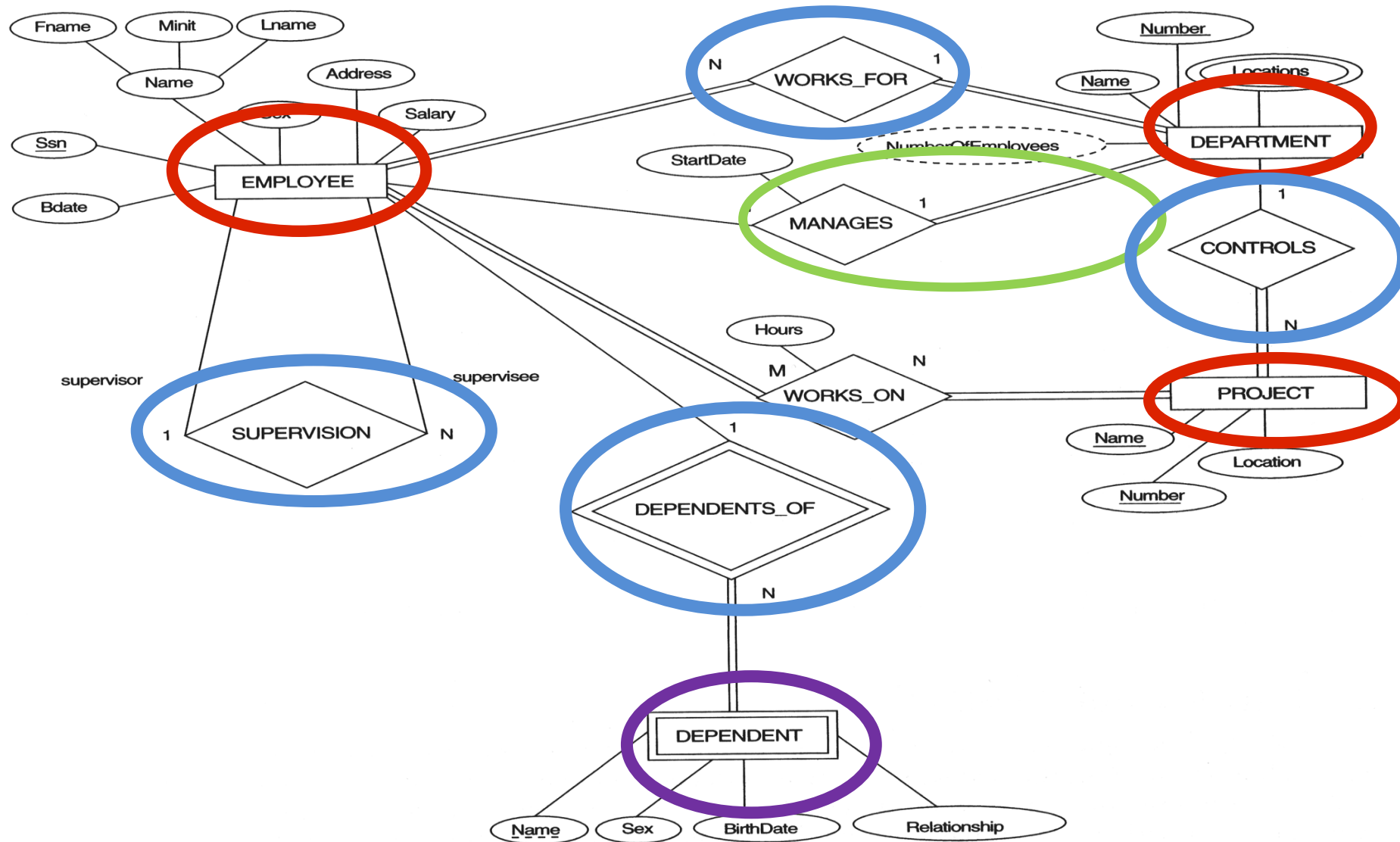
## 4. Mapping of Binary 1:N Relationship Types (cont)

### Example:

1:N relationship types WORKS\_FOR, CONTROLS, and SUPERVISION in the figure.

- For WORKS\_FOR we include the primary key DNUMBER of the DEPARTMENT relation as foreign key in the EMPLOYEE relation and call it DNO.
- For CONTROLS we include the primary key DNUMBER of the DEPARTMENT relation as foreign key in the PROJECT relation and call it DNUM.
- For SUPERVISION we include the primary key Ssn of the EMPLOYEE relation as foreign key in the EMPLOYEE relation and call it SuperSsn.

# 4. Mapping of Binary 1:N Relationship Types (cont)



## 4. Mapping of Binary 1:N Relationship Types (cont)

### EMPLOYEE

|            |       |       |       |       |         |     |        |          |     |
|------------|-------|-------|-------|-------|---------|-----|--------|----------|-----|
| <u>Ssn</u> | Fname | Minit | Lname | Bdate | Address | Sex | Salary | Superssn | Dno |
|------------|-------|-------|-------|-------|---------|-----|--------|----------|-----|

### DEPARTMENT

|                |       |        |              |
|----------------|-------|--------|--------------|
| <u>Dnumber</u> | Dname | Mgrssn | Mgrstartdate |
|----------------|-------|--------|--------------|

### PROJECT

|                |       |           |      |
|----------------|-------|-----------|------|
| <u>Pnumber</u> | Pname | Plocation | Dnum |
|----------------|-------|-----------|------|

### DEPENDENT

|             |                      |     |       |              |
|-------------|----------------------|-----|-------|--------------|
| <u>Essn</u> | <u>DependentName</u> | Sex | Bdate | Relationship |
|-------------|----------------------|-----|-------|--------------|

## 5. Mapping of Binary M:N Relationship Types.

- For each regular **binary M:N relationship** type R, *create a new relation* S to represent R.
- Include **as foreign key attributes** in S the primary keys of the relations that represent the participating entity types; *their combination will form the primary key* of S.
- Also **include any simple attributes** of the M:N relationship type (or simple components of composite attributes) as attributes of S.

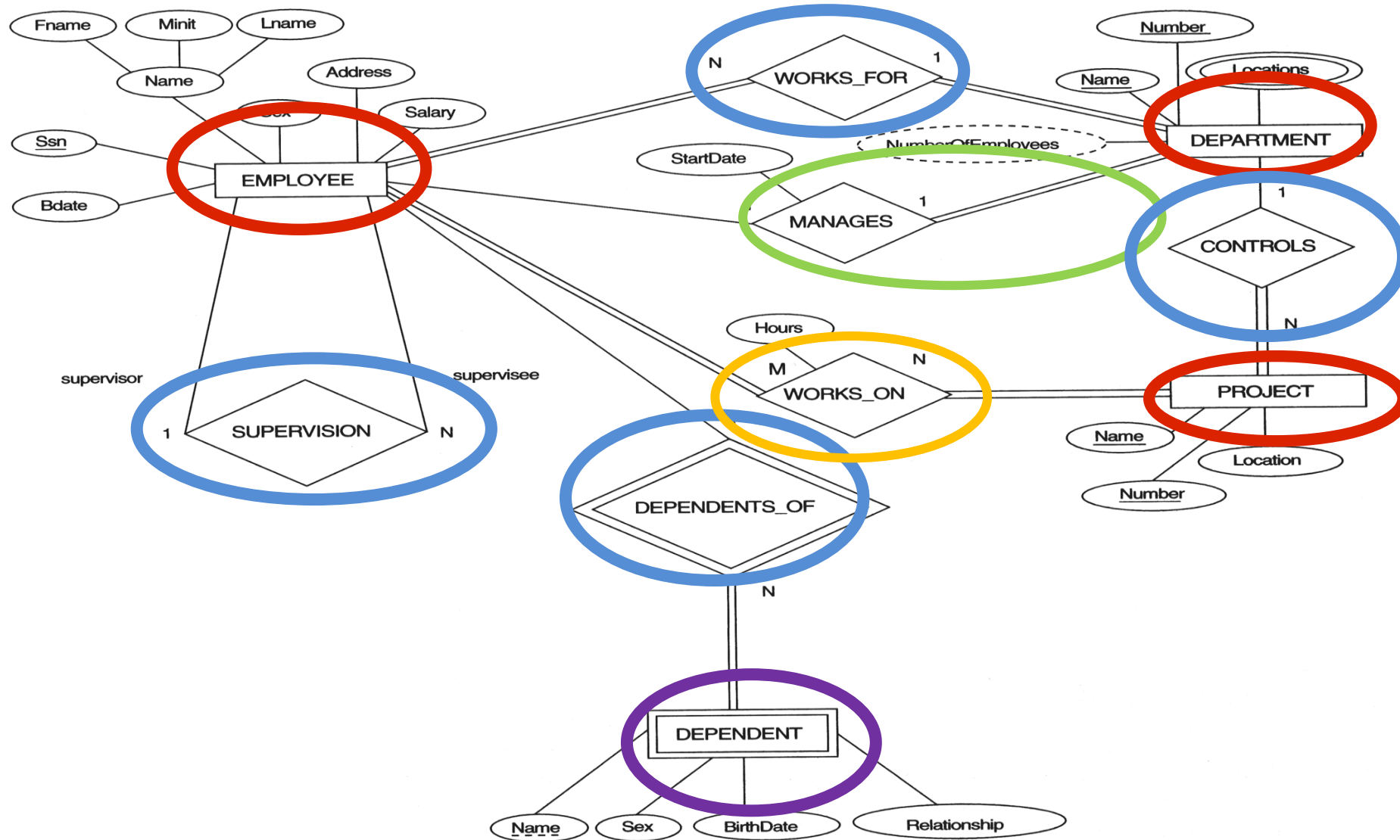
## 5. Mapping of Binary M:N Relationship Types (cont).

### Example:

The M:N relationship type **WORKS\_ON** from the ER diagram is mapped by creating a relation **WORKS\_ON** in the relational database schema.

- The primary keys of the **PROJECT** and **EMPLOYEE** relations are included as foreign keys in **WORKS\_ON** and renamed **PNO** and **ESSN**, respectively.
- Attribute **HOURS** in **WORKS\_ON** represents the **HOURS** attribute of the relation type.
- The primary key of the **WORKS\_ON** relation is the combination of the foreign key attributes {**ESSN**, **PNO**}.

# 5. Mapping of Binary M:N Relationship Types. (cont)



## 5. Mapping of Binary M:N Relationship Types (cont).

### EMPLOYEE

|            |       |       |       |       |         |     |        |          |     |
|------------|-------|-------|-------|-------|---------|-----|--------|----------|-----|
| <u>Ssn</u> | Fname | Minit | Lname | Bdate | Address | Sex | Salary | Superssn | Dno |
|------------|-------|-------|-------|-------|---------|-----|--------|----------|-----|

### DEPARTMENT

|                |       |        |              |
|----------------|-------|--------|--------------|
| <u>Dnumber</u> | Dname | Mgrssn | Mgrstartdate |
|----------------|-------|--------|--------------|

### PROJECT

|                |       |           |      |
|----------------|-------|-----------|------|
| <u>Pnumber</u> | Pname | Plocation | Dnum |
|----------------|-------|-----------|------|

### DEPENDENT

|             |                      |     |       |              |
|-------------|----------------------|-----|-------|--------------|
| <u>Essn</u> | <u>DependentName</u> | Sex | Bdate | Relationship |
|-------------|----------------------|-----|-------|--------------|

### WORKS\_ON

|             |            |       |
|-------------|------------|-------|
| <u>Essn</u> | <u>Pno</u> | Hours |
|-------------|------------|-------|

## 6. Mapping of Multivalued attributes.

- For each **multivalued attribute A**, create a new relation R. This relation R will include an attribute corresponding to A, plus the primary key attribute K-as a foreign key in R-of the relation that represents the entity type of relationship type that has A as an attribute.
- The **primary key of R is the combination** of A and K. If the multivalued attribute is composite, we include its simple components.

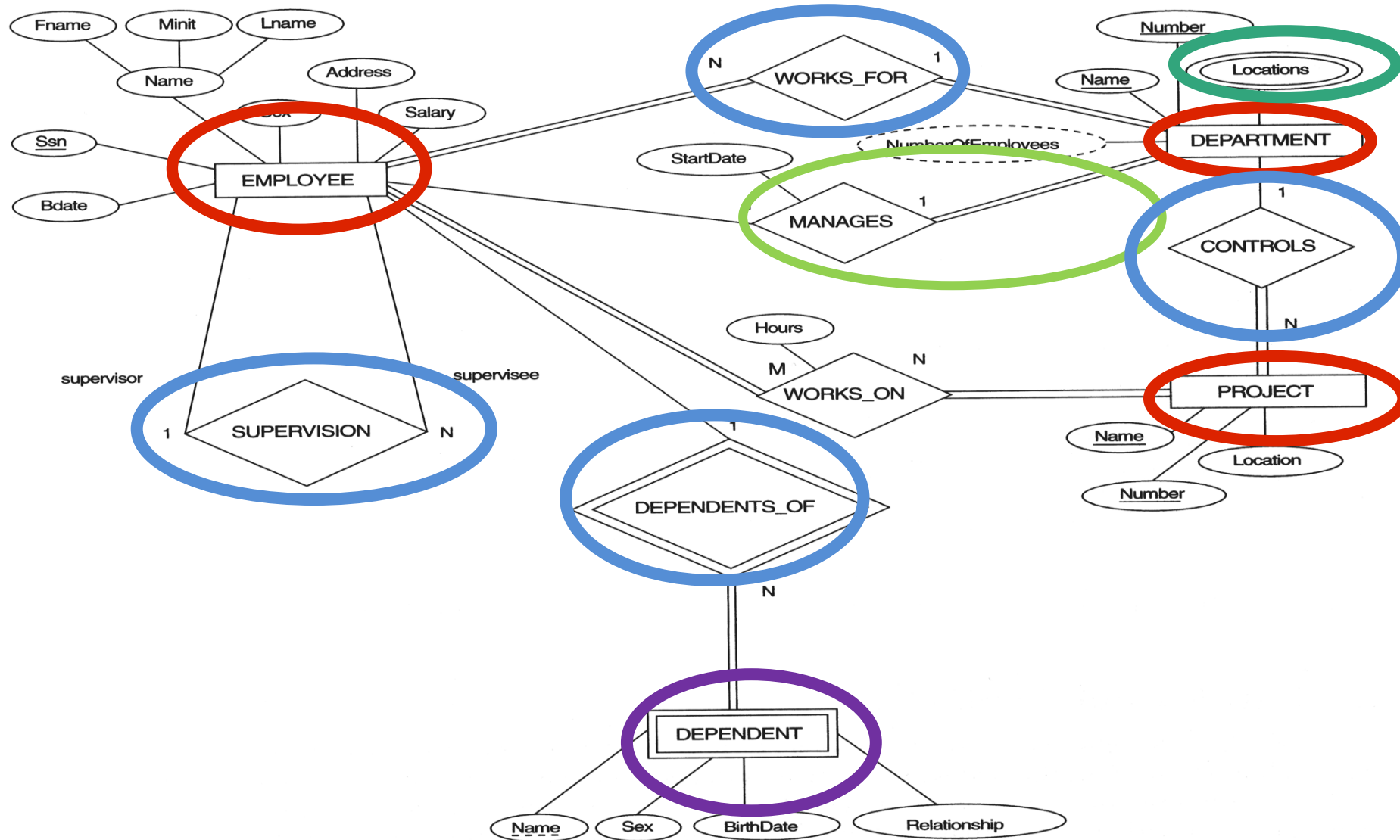
## 6. Mapping of Multivalued attributes (cont).

### Example:

The relation **DEPT\_LOCATIONS** is created.

- The attribute DLOCATION represents the multivalued attribute LOCATIONS of DEPARTMENT, while DNUMBER-as foreign key-represents the primary key of the DEPARTMENT relation.
- The primary key of R is the combination of {DNUMBER, DLOCATION}.

# 6. Mapping of Multivalued attributes (cont).



# 6. Mapping of Multivalued attributes (cont).

## EMPLOYEE

|            |       |       |       |       |         |     |        |          |     |
|------------|-------|-------|-------|-------|---------|-----|--------|----------|-----|
| <u>Ssn</u> | Fname | Minit | Lname | Bdate | Address | Sex | Salary | Superssn | Dno |
|------------|-------|-------|-------|-------|---------|-----|--------|----------|-----|

## DEPARTMENT

|                |       |        |              |
|----------------|-------|--------|--------------|
| <u>Dnumber</u> | Dname | Mgrssn | Mgrstartdate |
|----------------|-------|--------|--------------|

## PROJECT

|                |       |           |      |
|----------------|-------|-----------|------|
| <u>Pnumber</u> | Pname | Plocation | Dnum |
|----------------|-------|-----------|------|

## DEPENDENT

|             |                      |     |       |              |
|-------------|----------------------|-----|-------|--------------|
| <u>Essn</u> | <u>DependentName</u> | Sex | Bdate | Relationship |
|-------------|----------------------|-----|-------|--------------|

## WORKS\_ON

|             |            |       |
|-------------|------------|-------|
| <u>Essn</u> | <u>Pno</u> | Hours |
|-------------|------------|-------|

## DEPT\_LOCATION

|                |                  |
|----------------|------------------|
| <u>Dnumber</u> | <u>Dlocation</u> |
|----------------|------------------|

## 7. Mapping of N-ary Relationship Types.

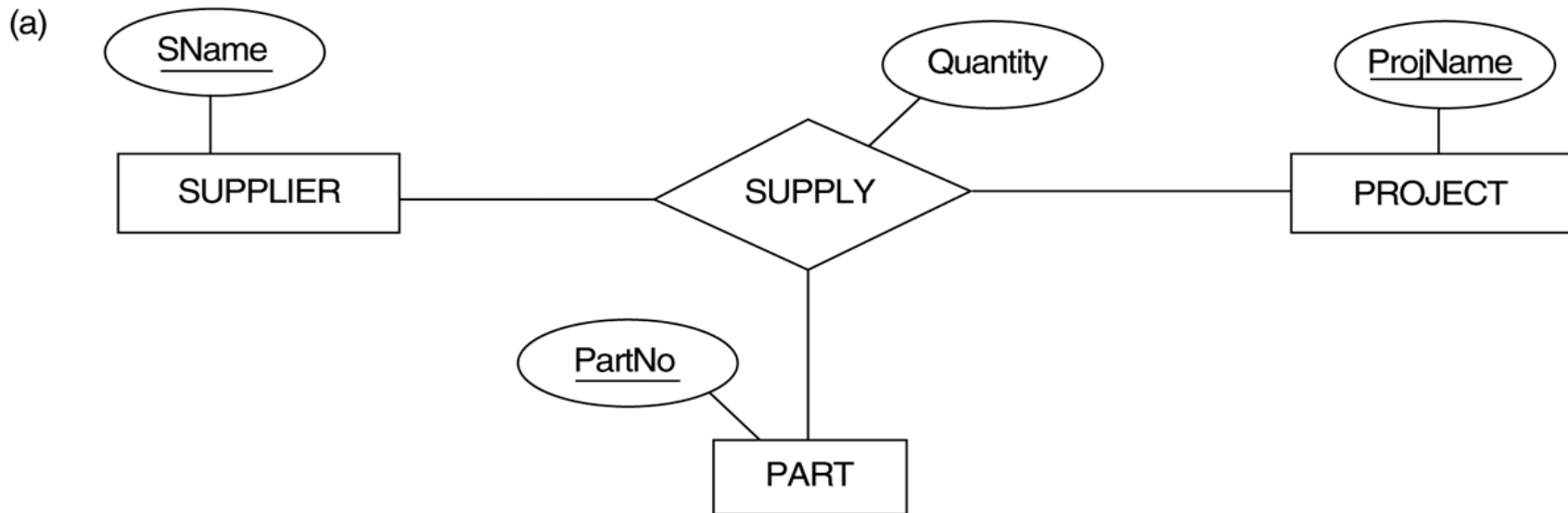
- For each **n-ary relationship** type R, **where  $n > 2$** , create a **new relationship** S to represent R.
- Include **as foreign key** attributes in S the primary keys of the relations that represent the participating entity types.
- Also **include any simple attributes** of the n-ary relationship type (or simple components of composite attributes) as attributes of S.

## 7. Mapping of N-ary Relationship Types (cont)

### Example:

The relationship type SUPPY in the ER below.

This can be mapped to the relation SUPPLY shown in the relational schema, whose primary key is the combination of the three foreign keys {SNAME, PARTNO, PROJNAME}



# 7. Mapping of N-ary Relationship Types (cont)

SUPPLIER

|              |     |
|--------------|-----|
| <u>SNAME</u> | ... |
|--------------|-----|

PROJECT

|                 |     |
|-----------------|-----|
| <u>PROJNAME</u> | ... |
|-----------------|-----|

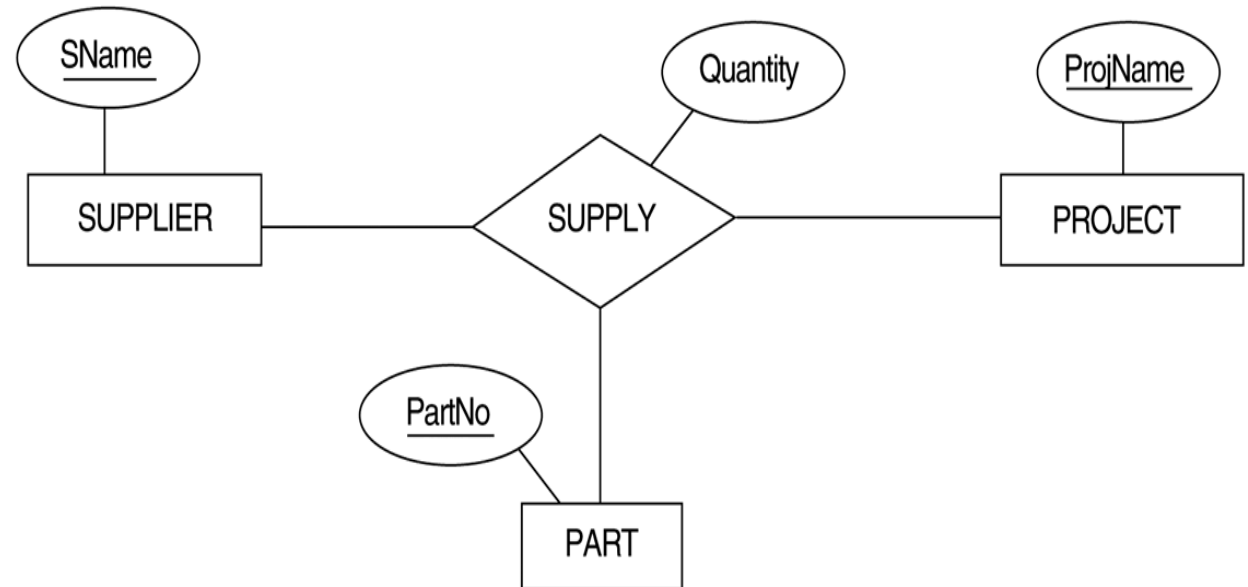
PART

|               |     |
|---------------|-----|
| <u>PARTNO</u> | ... |
|---------------|-----|

SUPPLY

|              |          |               |          |
|--------------|----------|---------------|----------|
| <u>SNAME</u> | PROJNAME | <u>PARTNO</u> | QUANTITY |
|--------------|----------|---------------|----------|

(a)



## 8. Options for Mapping Specialization or Generalization

There are 5 options.

- **Option 8A: Multiple relations-Superclass and subclasses.**
- **Option 8B: Multiple relations-Subclass relations only.**
- **Option 8C: Single relation with one type attribute.**
- **Option 8D: Single relation with multiple type attributes.**
- **Option 8E: Mapping of Shared Subclasses (Multiple Inheritance)**

## 8. Options for Mapping Specialization or Generalization (cont)

### Option 8A: **Multiple relations-Superclass and subclasses.**

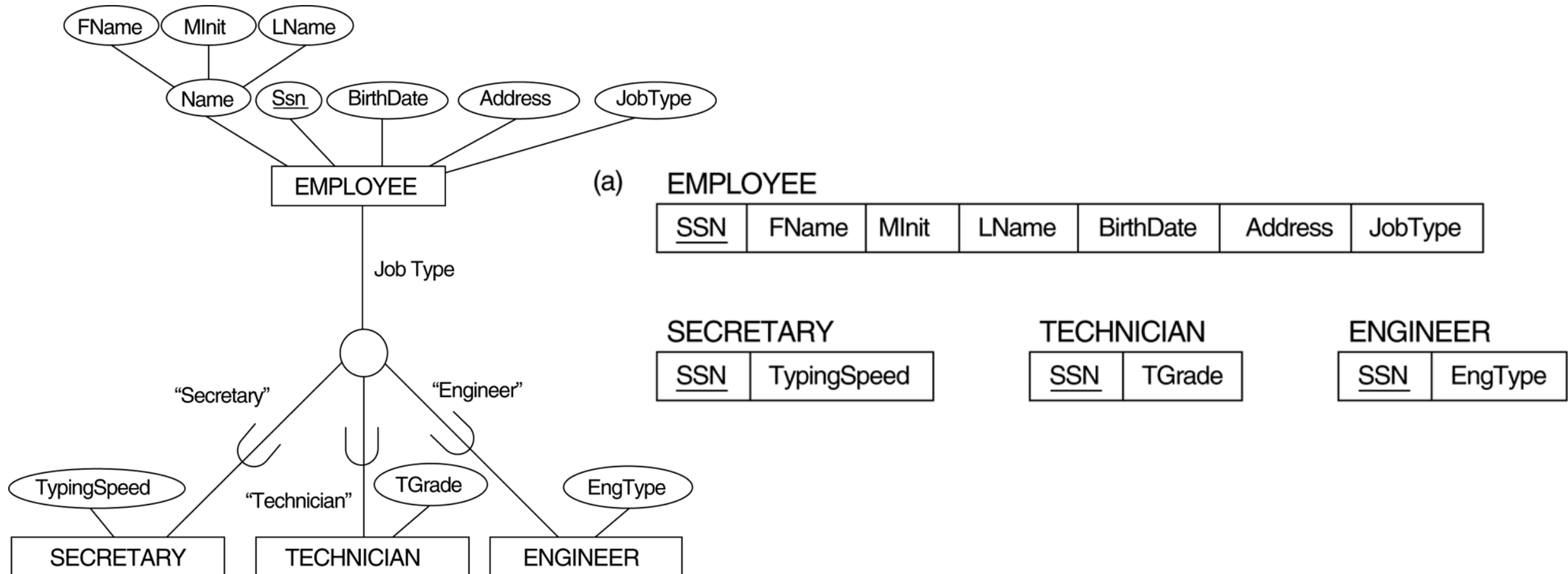
**Create a relation**  $L$  for  $C$  with attributes  $\text{Attrs}(L) = \{k, a_1, \dots, a_n\}$  and  $\text{PK}(L) = k$ .

**Create a relation**  $L_i$  for each subclass  $S_i$ ,  $1 < i < m$ , with the attributes  $\text{Attrs}(L_i) = \{k\} \cup \{\text{attributes of } S_i\}$  and  $\text{PK}(L_i) = k$ .

This option works **for any specialization** (total or partial, disjoint or over-lapping).

# 8. Options for Mapping Specialization or Generalization (cont)

## Option 8A: Multiple relations-Superclass and subclasses.



## 8. Options for Mapping Specialization or Generalization (cont)

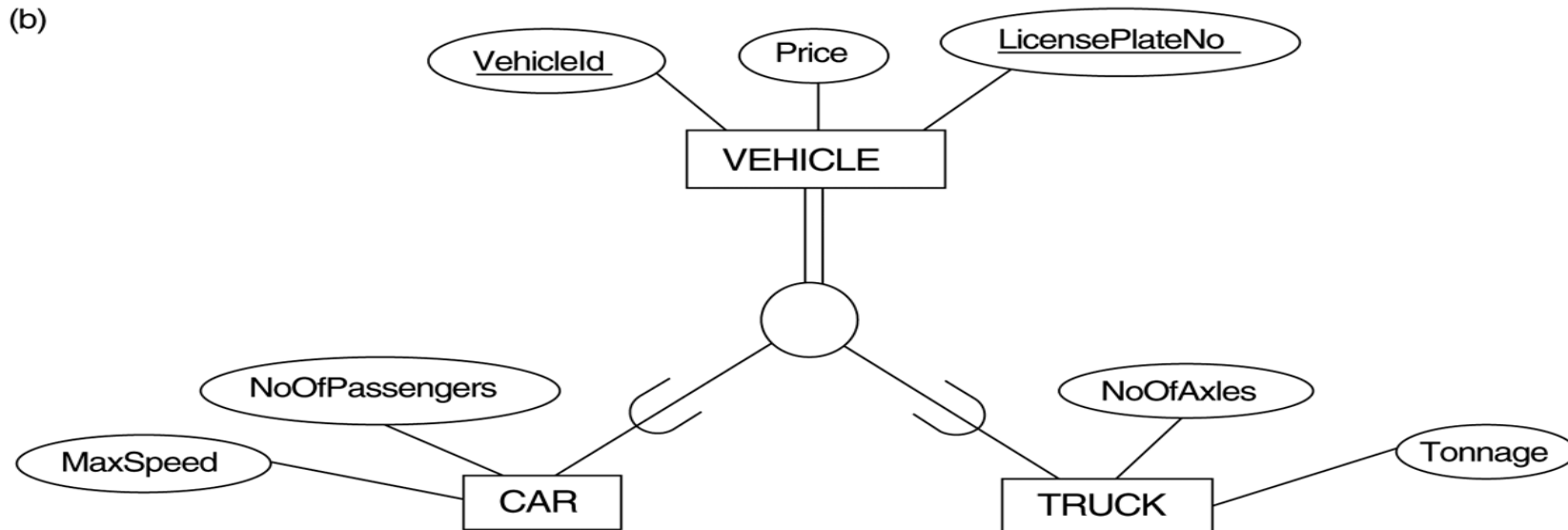
### Option 8B: Multiple relations-Subclass relations only

Create a relation  $L_i$  for each subclass  $S_i$ ,  $1 < i < m$ , with the attributes  $\text{Attr}(L_i) = \{\text{attributes of } S_i\} \cup \{k, a_1, \dots, a_n\}$  and  $\text{PK}(L_i) = k$ .

This option only works for a specialization whose subclasses are **total** (every entity in the superclass must belong to (at least) one of the subclasses).

# 8. Options for Mapping Specialization or Generalization (cont)

## Option 8B: Multiple relations-Subclass relations only



(b) CAR

|                  |                |       |          |                |
|------------------|----------------|-------|----------|----------------|
| <u>VehicleId</u> | LicensePlateNo | Price | MaxSpeed | NoOfPassengers |
|------------------|----------------|-------|----------|----------------|

TRUCK

|                  |                |       |           |  |
|------------------|----------------|-------|-----------|--|
| <u>VehicleId</u> | LicensePlateNo | Price | NoOfAxles |  |
|------------------|----------------|-------|-----------|--|

## 8. Options for Mapping Specialization or Generalization (cont)

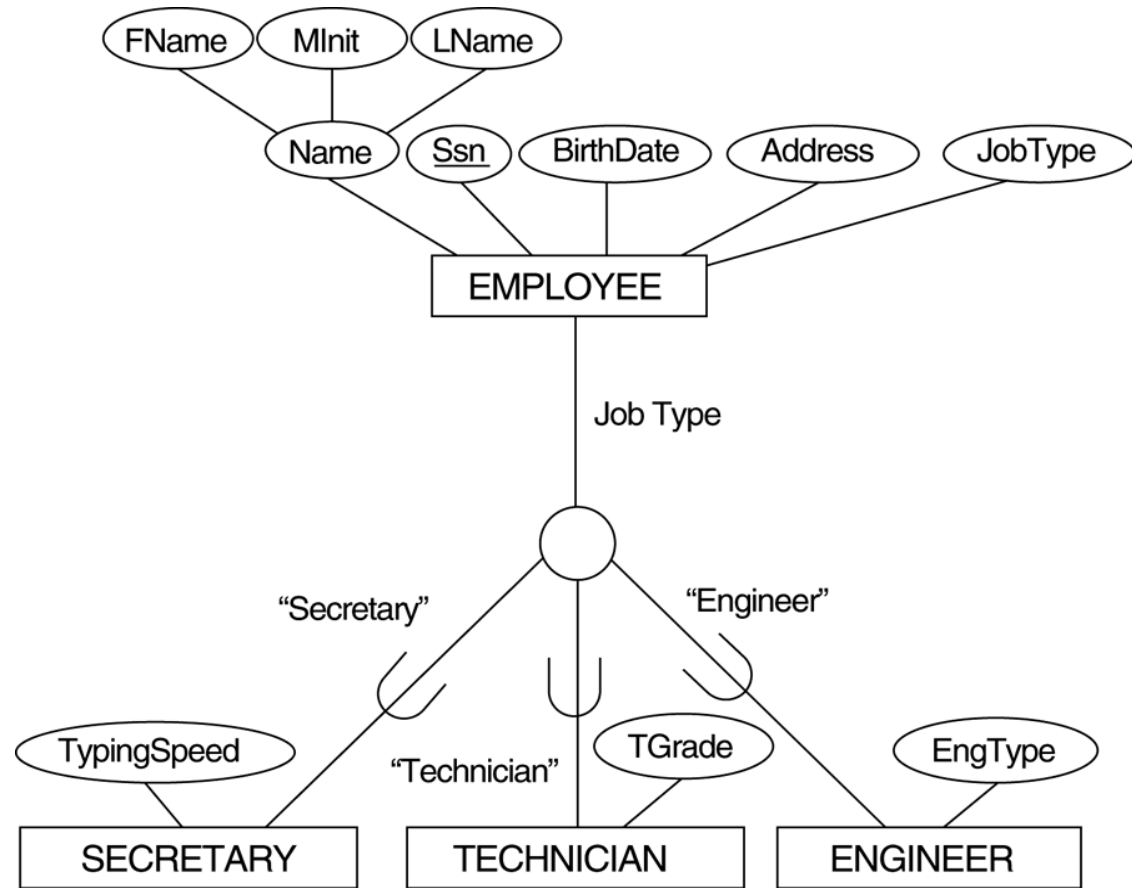
**Option 8C: Single relation with one type attribute.**

Create a single relation  $L$  with attributes  $\text{Attrs}(L) = \{k, a_1, \dots, a_n\} \cup \{\text{attributes of } S_1\} \cup \dots \cup \{\text{attributes of } S_m\} \cup \{t\}$  and  $\text{PK}(L) = k$ .

The attribute  $t$  is called a type (or **discriminating**) attribute that indicates the subclass to which each tuple belongs

# 8. Options for Mapping Specialization or Generalization (cont)

**Option 8C: Single relation with one type attribute.**



(c) EMPLOYEE

| <u>SSN</u> | FName | MInit | LName | BirthDate | Address | JobType | TypingSpeed | TGrade |  |
|------------|-------|-------|-------|-----------|---------|---------|-------------|--------|--|
|------------|-------|-------|-------|-----------|---------|---------|-------------|--------|--|

## 8. Options for Mapping Specialization or Generalization (cont)

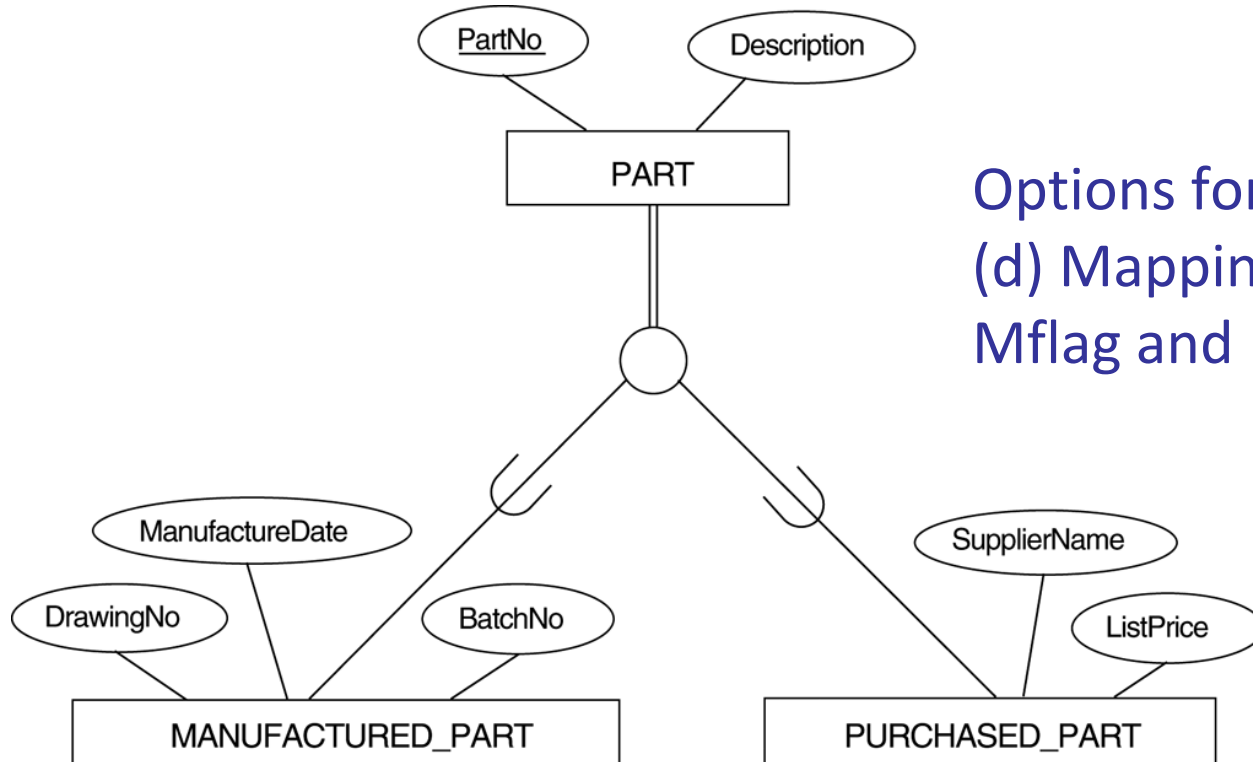
**Option 8D: Single relation with multiple type attributes.**

Create a single relation schema  $L$  with attributes  $\text{Attrs}(L) = \{k, a_1, \dots, a_n\} \cup \{\text{attributes of } S_1\} \cup \dots \cup \{\text{attributes of } S_m\} \cup \{t_1, t_2, \dots, t_m\}$  and  $\text{PK}(L) = k$ .

Each  $t_i$ ,  $1 < i < m$ , is a Boolean type attribute indicating whether a tuple belongs to the subclass  $S_i$ .

# 8. Options for Mapping Specialization or Generalization (cont)

## Option 8D: Single relation with multiple type attributes.



Options for mapping specialization or generalization.  
(d) Mapping using option 8D with Boolean type fields Mflag and Pflag.

(d) PART

|               |             |       |           |                 |         |       |              |           |
|---------------|-------------|-------|-----------|-----------------|---------|-------|--------------|-----------|
| <u>PartNo</u> | Description | MFlag | DrawingNo | ManufactureDate | BatchNo | PFlag | SupplierName | ListPrice |
|---------------|-------------|-------|-----------|-----------------|---------|-------|--------------|-----------|

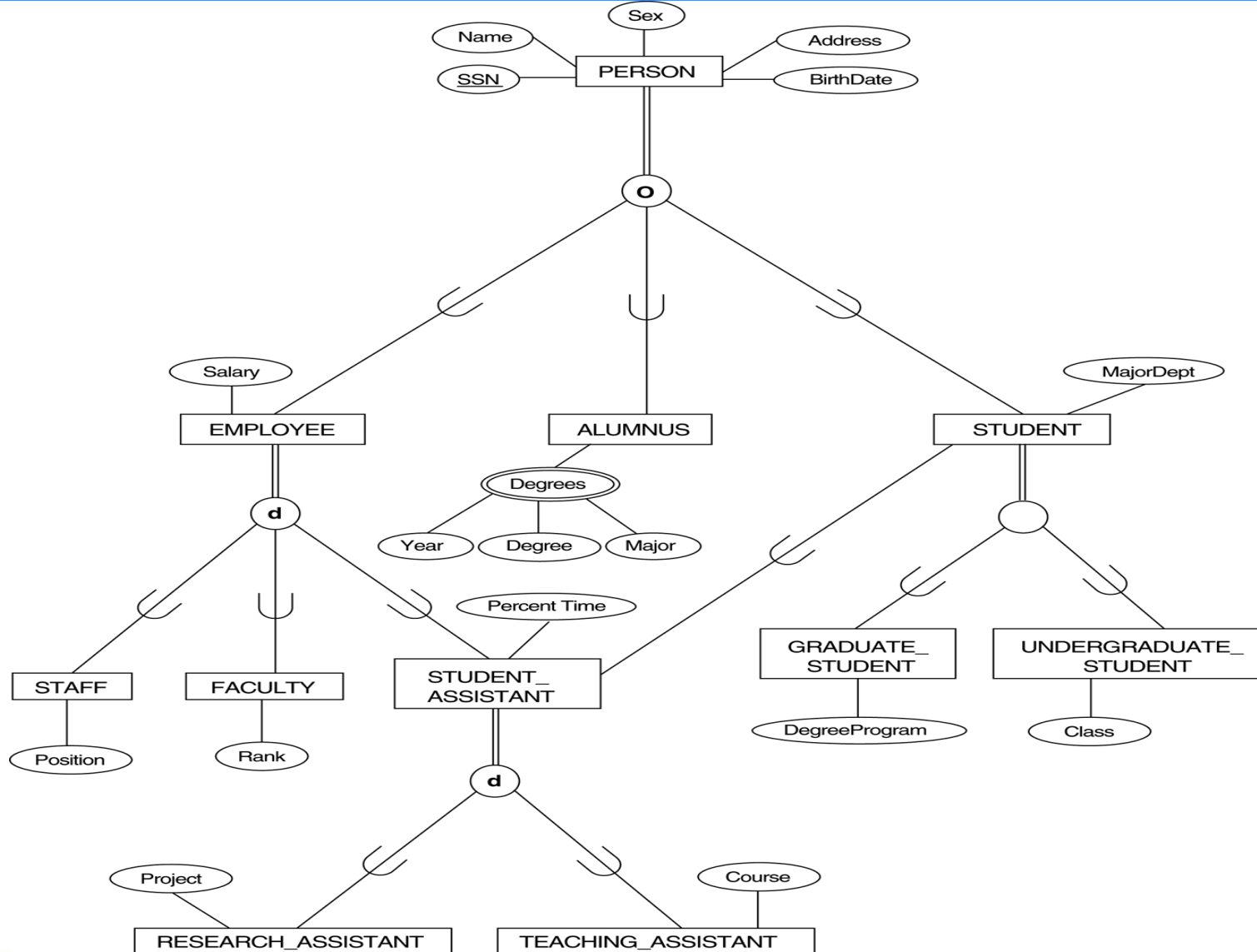
## 8. Options for Mapping Specialization or Generalization (cont)

### Option 8E: **Mapping of Shared Subclasses** (Multiple Inheritance)

A **shared subclass**, such as **STUDENT\_ASSISTANT**, is a subclass of several classes, indicating multiple inheritance. These classes must all have the same key attribute; otherwise, the shared subclass would be modeled as a category.

We **can apply any of the options** discussed in Step 8 to a shared subclass, subject to the restriction discussed in Step 8 of the mapping algorithm. Below both 8C and 8D are used for the shared class **STUDENT\_ASSISTANT**.

# 8. Options for Mapping Specialization or Generalization (cont)



## 8. Options for Mapping Specialization or Generalization (cont)

### PERSON

|            |      |           |     |         |
|------------|------|-----------|-----|---------|
| <u>SSN</u> | Name | BirthDate | Sex | Address |
|------------|------|-----------|-----|---------|

### EMPLOYEE

|            |        |              |          |      |             |        |        |         |  |
|------------|--------|--------------|----------|------|-------------|--------|--------|---------|--|
| <u>SSN</u> | Salary | EmployeeType | Position | Rank | PercentTime | RAFlag | TAFlag | Project |  |
|------------|--------|--------------|----------|------|-------------|--------|--------|---------|--|

### ALUMNUS

|            |
|------------|
| <u>SSN</u> |
|------------|

### ALUMNUS\_DEGREES

|            |      |        |  |
|------------|------|--------|--|
| <u>SSN</u> | Year | Degree |  |
|------------|------|--------|--|

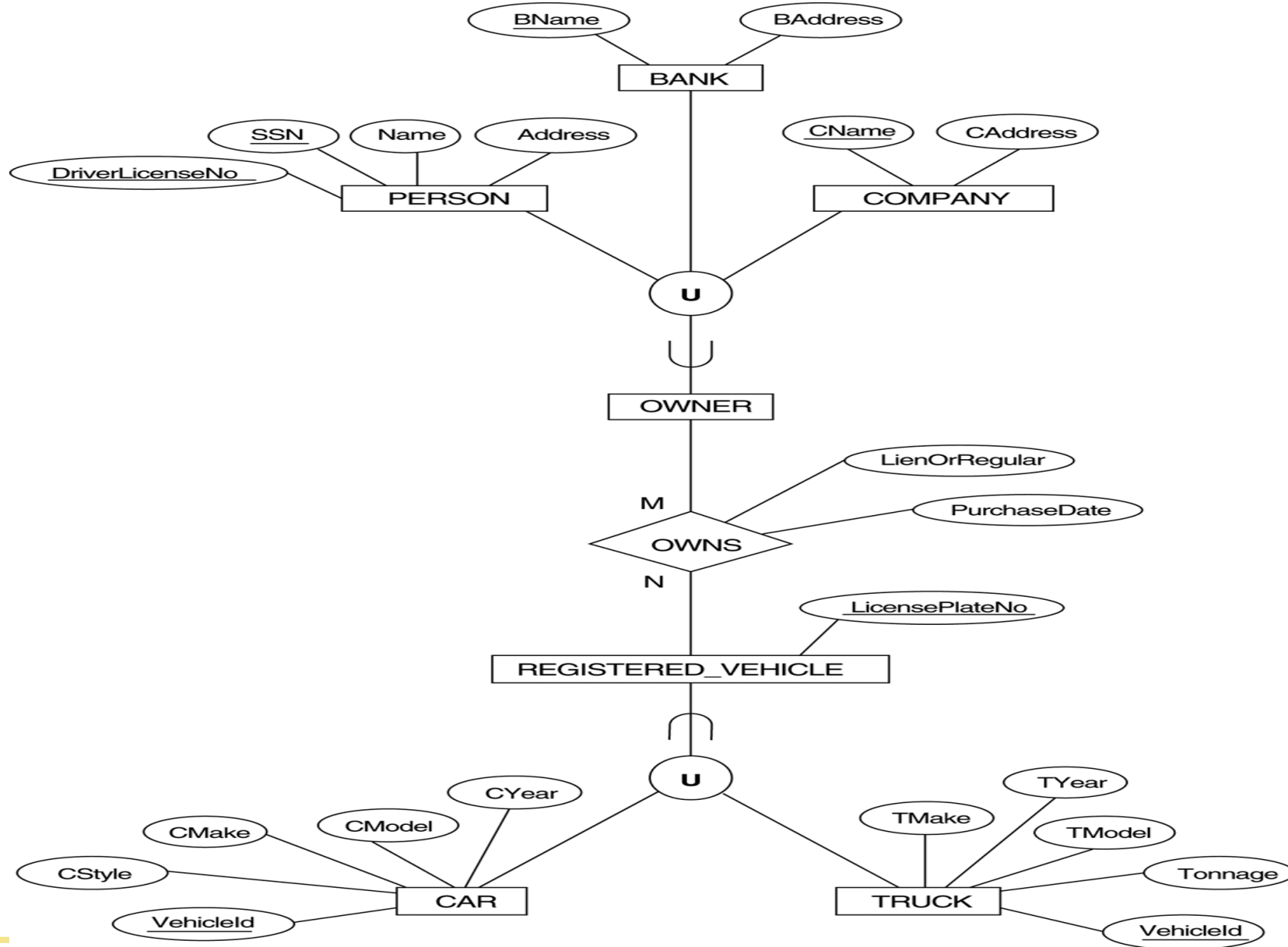
### STUDENT

|            |           |          |               |               |       |                |
|------------|-----------|----------|---------------|---------------|-------|----------------|
| <u>SSN</u> | MajorDept | GradFlag | UndergradFlag | DegreeProgram | Class | StudAssistFlag |
|------------|-----------|----------|---------------|---------------|-------|----------------|

## 9. Mapping of Union Types (Categories).

- For **mapping a category** whose defining **superclass have different keys**, it is customary to specify a new key attribute, called **a surrogate key**, when creating a relation to correspond to the category.
- In the example below we can create a relation **OWNER** to correspond to the OWNER category and include any attributes of the category in this relation. The primary key of the OWNER relation is the surrogate key, which we called **OwnerId**.

# 9. Mapping of Union Types (cont)



## 9. Mapping of Union Types (cont)

### PERSON

|            |                 |      |         |  |
|------------|-----------------|------|---------|--|
| <u>SSN</u> | DriverLicenseNo | Name | Address |  |
|------------|-----------------|------|---------|--|

### BANK

|              |          |         |
|--------------|----------|---------|
| <u>BName</u> | BAddress | OwnerId |
|--------------|----------|---------|

### COMPANY

|              |          |         |
|--------------|----------|---------|
| <u>CName</u> | CAddress | OwnerId |
|--------------|----------|---------|

### OWNER

|                |
|----------------|
| <u>OwnerId</u> |
|----------------|

### REGISTERED\_VEHICLE

|                  |                    |
|------------------|--------------------|
| <u>VehicleId</u> | LicensePlateNumber |
|------------------|--------------------|

### CAR

|                  |        |       |        |  |
|------------------|--------|-------|--------|--|
| <u>VehicleId</u> | CStyle | CMake | CModel |  |
|------------------|--------|-------|--------|--|

### TRUCK

|                  |       |        |         |       |
|------------------|-------|--------|---------|-------|
| <u>VehicleId</u> | TMake | TModel | Tonnage | TYear |
|------------------|-------|--------|---------|-------|

### OWNS

|                |                  |              |               |
|----------------|------------------|--------------|---------------|
| <u>OwnerId</u> | <u>VehicleId</u> | PurchaseDate | LienOrRegular |
|----------------|------------------|--------------|---------------|

# Summary

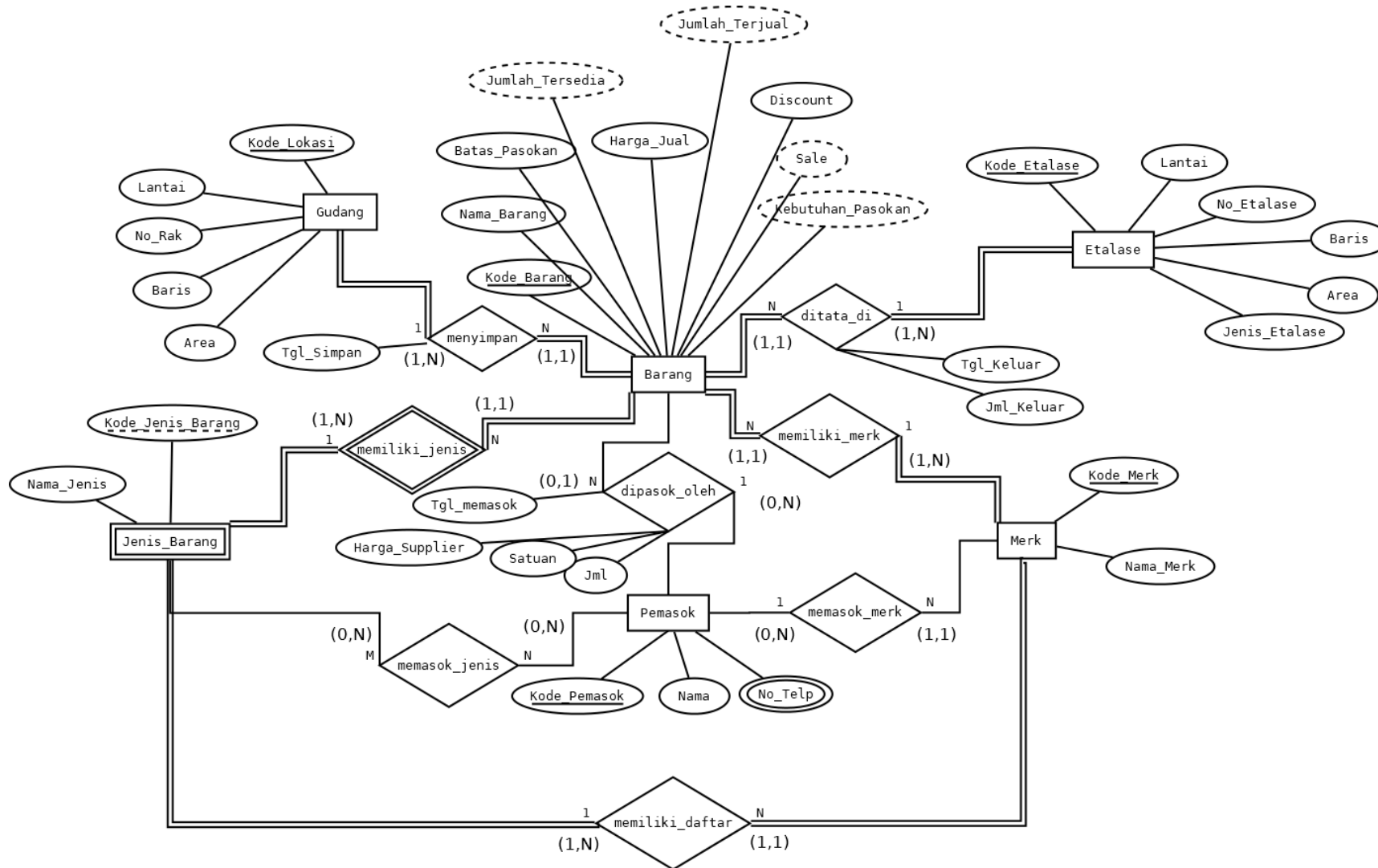
## A. ER to Realtional Model

1. Mapping of Regular Entity Types: (create table)
2. Mapping of Weak Entity Types: (create table)
3. Mapping of Binary 1:1 Relationship Types: (update/create table if all partial)
4. Mapping of Binary 1:N Relationship Types: (update table with attribute)
5. Mapping of Binary M:N Relationship Types: (create table with attribute)
6. Mapping of Multivalued attributes: (create table)
7. Mapping of N-ary Relationship Types: (create table)

## B. E-ER to Relational Model

8. Options for Mapping Specialization or Generalization  
(create table with optional attribute)
9. Mapping of Union Types (Categories).  
(create table with optional attribute)

# Exercise





Barangsiapa yang berangkat menimba ilmu untuk mengamalkan ilmu,  
niscaya ilmu yang sedikit pun akan bermanfaat baginya